

MIT Academy of Engineering, Alandi(D), Pune
Department of Computer Engineering

Assignment No .2

- * Student Name:** Sohan Jarag
- PRN Number:** 202501040209
- Division / Batch:** CS2/C25
- Dataset Name & Source:** Movie review from Kaggle

Problem statement 1: A streaming platform wants to know overall content quality. Find average rating.

```
import numpy as np

avg_rating = np.mean(df['rating'])

print(avg_rating)
```

Explanation:

This calculates the mean rating of all movies using **NumPy**. It helps understand the overall quality of movies in the dataset. A higher value indicates better audience satisfaction.

Problem statement 2: Identify the best movie to recommend to premium users.

```
idx = df['rating'].idxmax()

top_movie = df.loc[idx]

print(top_movie)
```

Explanation:

This identifies the movie with the maximum rating using **Pandas indexing**. It helps platforms recommend the best content. Useful for highlighting top-performing movies.

Problem 3 : Analyze modern-era movie production trends.

```
recent = df[df['year'] > 2000]
count = recent.shape[0]
print(count)
```

Explanation:

This filters movies based on year and counts them. It helps analyze modern movie trends. Shows how the industry has grown in recent years.

Problem 4: Calculate total money invested in movie production

```
df['budget'] = pd.to_numeric(df['budget'], errors='coerce')
total_budget = np.sum(df['budget'])
print(total_budget)
```

Explanation :

This sums all movie budgets using **NumPy**. It reflects the total money spent on movie production. Useful for understanding industry scale.

Problem 5: Find most commercially successful movies.

```
df['box_office'] = pd.to_numeric(df['box_office'], errors='coerce')
top5 = df.sort_values(by='box_office', ascending=False).head()
print(top5)
```

Explanation :

This sorts movies by box office and selects top 5. It helps identify blockbuster movies. Useful for analyzing commercial success.

Problem 6: Determine average movie length for scheduling.

```
df['run_time'] = df['run_time'].str.extract('(\d+)').astype(float)
avg_time = df['run_time'].mean()
print(avg_time) Problem
```

Explanation :

This computes the average duration of movies. It helps streaming platforms plan schedules. Shows typical movie length.

Problem 7: A platform wants to focus on popular genres like Drama.

```
drama = df[df['genre'].str.contains('Drama', na=False)]  
print(drama.shape[0])
```

Explanation :

This counts movies belonging to the Drama genre. Helps identify genre popularity. Useful for content planning.

Problem 8 : Identify top-quality movies for featured section

```
high = df[df['rating'] > 8.5]  
print(high['name'])
```

Exlanation:

This filters movies with rating above 8.5. It helps find premium-quality content. Useful for recommendations.

Problem 9: Analyze yearly production trends.

```
year_count = df['year'].value_counts()  
print(year_count)
```

Explanation :

This counts how many movies were released each year. It shows production trends over time. Useful for trend analysis.

Problem 10: Identify most produced genre.

```
genres = df['genre'].str.split(',').explode()  
common = genres.value_counts().idxmax()  
print(common)
```

Explanation :

This finds the most frequent genre in dataset. It helps understand audience preference. Useful for production strategy

Problem 11: Check if higher investment leads to higher returns.

```
corr = df[['budget', 'box_office']].corr()  
print(corr)
```

Explanation :

This checks relationship between budget and revenue. It shows whether higher investment leads to higher returns. Important for financial decisions.

Problem 12: Identify incomplete records affecting analysis.

```
missing = df.isnull().sum()  
print(missing)
```

Explanation :

This identifies missing values in dataset. Helps improve data quality before analysis. Ensures accurate results.

Problem 13: Find central tendency without outliers.

```
import numpy as np  
median = np.median(df['rating'])  
print(median)
```

Explanation :

This calculates the middle value of ratings using **NumPy**. It reduces effect of outliers. Gives a more stable central value.

Problem 14: Identify yearly top performers

```
idx = df.groupby('year')['box_office'].idxmax()  
result = df.loc[idx]  
print(result)
```

Explanation :

This finds the top-earning movie for each year. Helps identify yearly winners. Useful for historical analysis.

Problem 15: Measure diversity in movie direction

```
unique_dir = df['directors'].nunique()  
print(unique_dir)
```

Explanation :

This counts distinct directors in dataset. Shows diversity in filmmaking. Useful for industry insights.

Problem 16 : Analyze long-duration content trend.

```
long_movies = df[df['run_time'] > 120]  
print(long_movies.shape[0])
```

Explanation :

This filters movies with runtime above 120 minutes. Helps analyze long-duration content. Useful for scheduling decisions

Problem 17: Find best movie in every genre category.

```
idx = df.groupby('genre')['rating'].idxmax()  
top_genre = df.loc[idx]  
print(top_genre)
```

Explanation :

This finds highest-rated movie for every genre. Helps categorize best content. Useful for genre-based recommendations.

Problem 18: Check consistency of movie quality.

```
std = np.std(df['rating'])  
print(std)
```

Explanation :

This calculates standard deviation using **NumPy**. It measures how spread out ratings are. Higher value means inconsistent quality.

Problem 19: Identify high-investment productions.

```
avg = df['budget'].mean()
above = df[df['budget'] > avg]
print(above['name'])
```

Explanation :

This filters movies with budget greater than average. Helps identify high-investment films. Useful for financial analysis.

Problem 20 : Find movies with best return on investment.

```
df['profit_ratio'] = df['box_office'] / df['budget']
top_profit = df.sort_values(by='profit_ratio', ascending=False).head()
print(top_profit)
```

Explanation :

This calculates profit ratio (box office / budget). Helps find movies with best return on investment. Useful for business decisions

